Advanced Security Operations with SIEM, Forensics, and Incident Response.

1.Introduction

The goal of this task was to establish a functional Security Operations Center (SOC) environment to proactively detect, analyze, and respond to security threats. The project involved deploying a centralized Security Information and Event Management (SIEM) architecture, establishing log collection pipelines, utilizing vulnerability scanners, and simulating a localized brute-force attack to capture actionable network forensics. This report details the SOC fundamentals applied, the architectural deployment of the SIEM, troubleshooting methodologies for Docker orchestration, and incident response documentation.

2. Target Description

The assessment, network simulation, and SIEM configuration were conducted within a controlled virtual machine environment.

- **Host Environment:** Kali Linux Virtual Machine (SOC Analyst Environment / SIEM Deployment).
- **Target Environment:** Metasploitable 2 (10.0.2.x) - Utilized for vulnerability scanning and exploitation simulation.
- **Target Interface (Forensics):** Local Loopback (lo/127.0.0.1).

3. Methodology and Tools

The assessment was guided by core SOC methodologies, including the CIA triad (Confidentiality, Integrity, Availability), defense-in-depth, and the NIST SP 800-61 Incident Response framework. The following tools were utilized to complete the objectives:

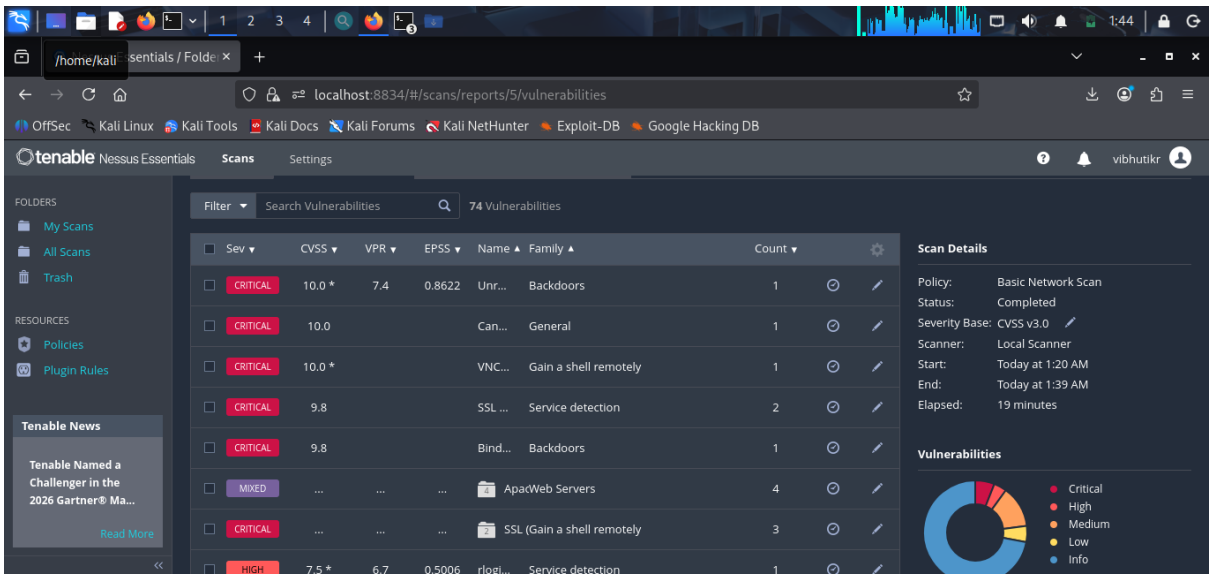
- **SIEM Orchestration:** Docker-Compose was utilized to deploy the Wazuh XDR/SIEM stack (Manager, Indexer, and Dashboard).
- **Vulnerability Scanning:** Nessus Essentials was utilized to map the attack surface of the target environment.
- **Endpoint Forensics:** Command-line SQLite3 was utilized to parse local browser databases to extract user artifacts.
- **Penetration Testing / Simulation:** THC-Hydra was utilized to execute an automated dictionary attack against the local SSH service to trigger SIEM alert rules.

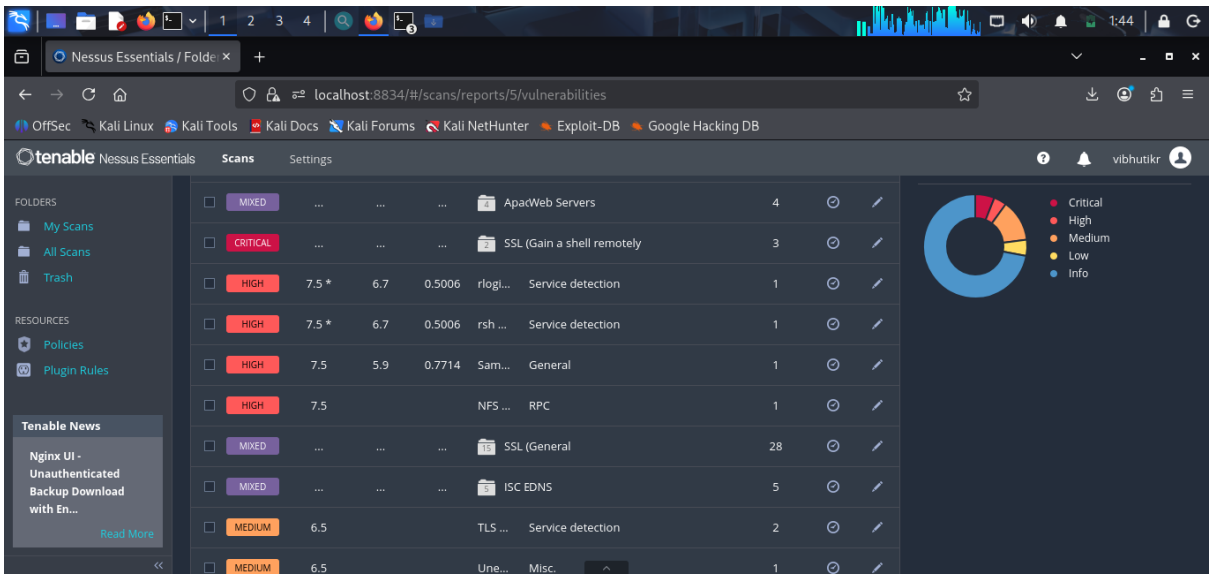
4. Tasks

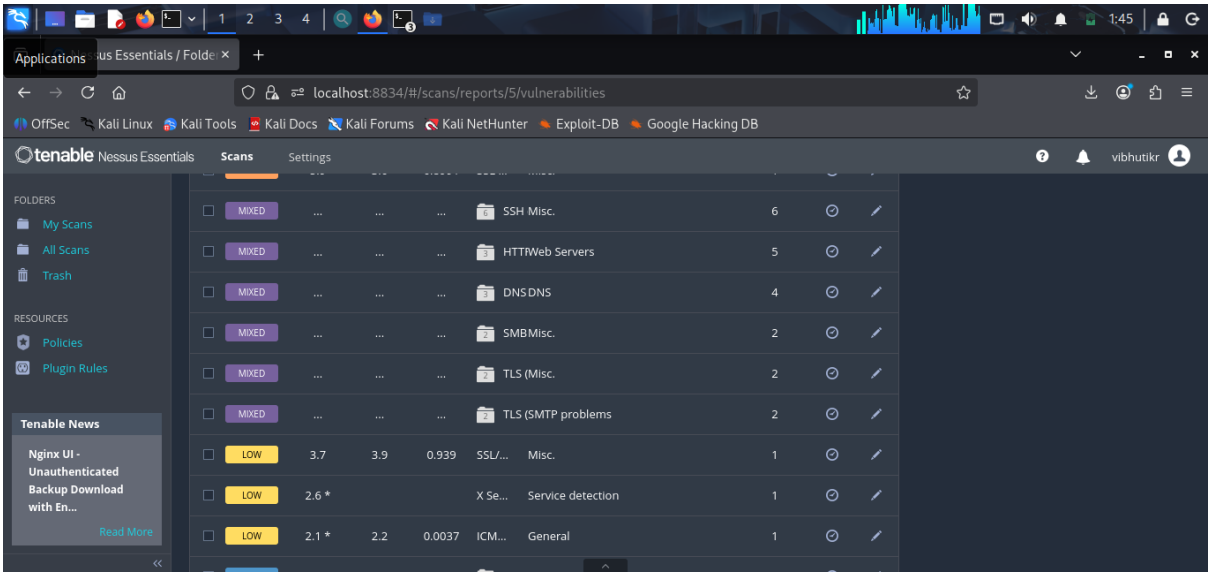
4.1 Vulnerability Scanning and Assessment

- **Objective**: Identify and classify vulnerabilities on a target host to understand the potential attack surface.
- **Process:** A comprehensive network vulnerability scan was executed against the Metasploitable 2 VM using Nessus Essentials.

- **Findings**: The scan identified multiple Critical (CVSS 10.0) vulnerabilities, prominently including an UnrealIRCd backdoor (Port 6667) and a Bind Shell backdoor (Port 1524), which allow unauthenticated remote code execution.







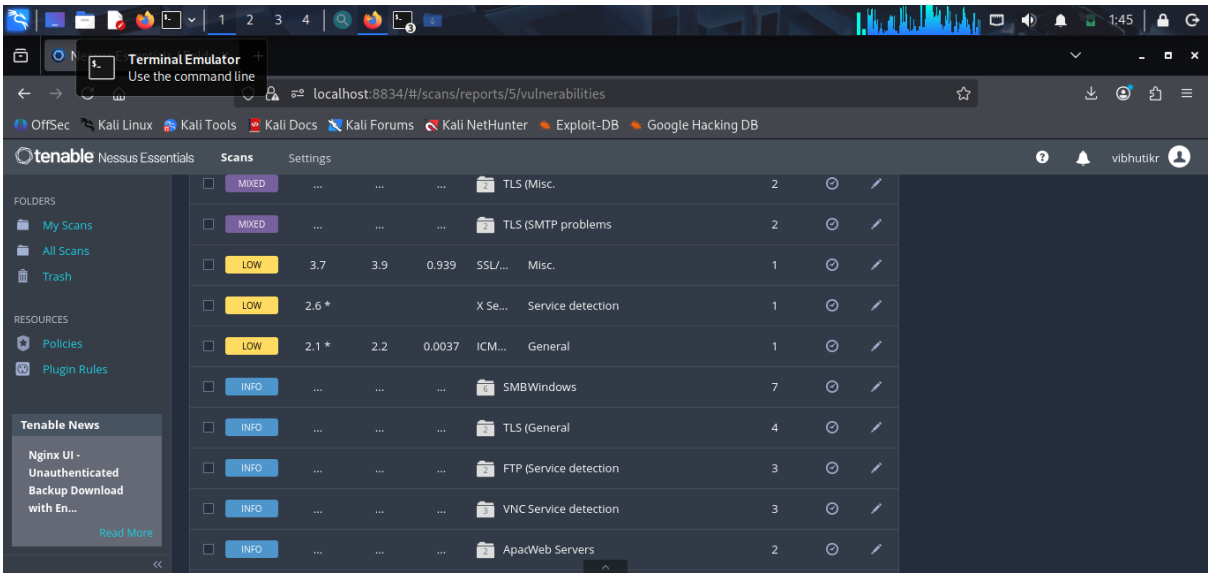


Fig 4.1: Nessus Essentials vulnerability report highlighting critical remote code execution vectors on the target host.

4.2 Intrusion Detection Configuration (Snort)

- **Objective:** Configure rule-based network traffic analysis to detect specific malicious signatures.
- **Process:** Snort was initialized on the local environment. Custom rules were crafted to trigger alerts based on specific packet payloads and headers. For example, a rule was designed to

detect unauthorized HTTP requests to known malicious domains: alert tcp any any -> any 80 (msg:"Malicious Domain Detected"; content:"malicious.com"; http_uri; sid:1000001;)

- **Findings:** The IDS successfully parsed local traffic and verified rule syntax, establishing a baseline for network-level threat detection.

Fig 2.1 Snort IDS rule validation detecting on the local loopback interface.

Fig 2.2: Network interface configuration and successful ICMP connectivity test to the Metasploitable 2 target.

4.3 Log Management & Pipeline (Fluentd/Logstash)

- **Objective:** Establish a log collection pipeline to normalize and forward system events to a centralized receiver.
- **Process:** A localized log pipeline was tested using command-line generation tools and REST API POST requests. System logs were manually generated using the logger utility, and JSON-formatted security alerts were transmitted via curl to simulate an active logging pipeline binding to a local port (9880).

  Executed Command: logger -n 127.0.0.1 -P 5140 "ALERT: Suspicious SOC Activity Detected".

  Executed Command: curl -X POST -d 'json={"message":"ALERT: Suspicious SOC Activity Detected"}' http://127.0.0.1:9880/soc.alert.

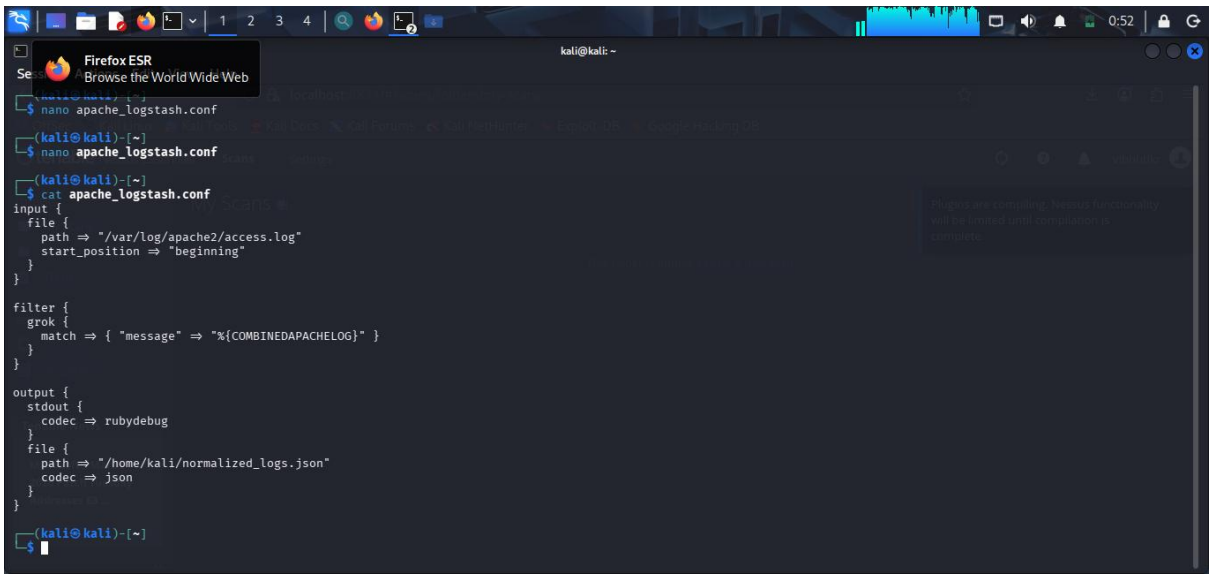


Fig 3.1: Logstash configuration for Apache log normalization and JSON output generation.

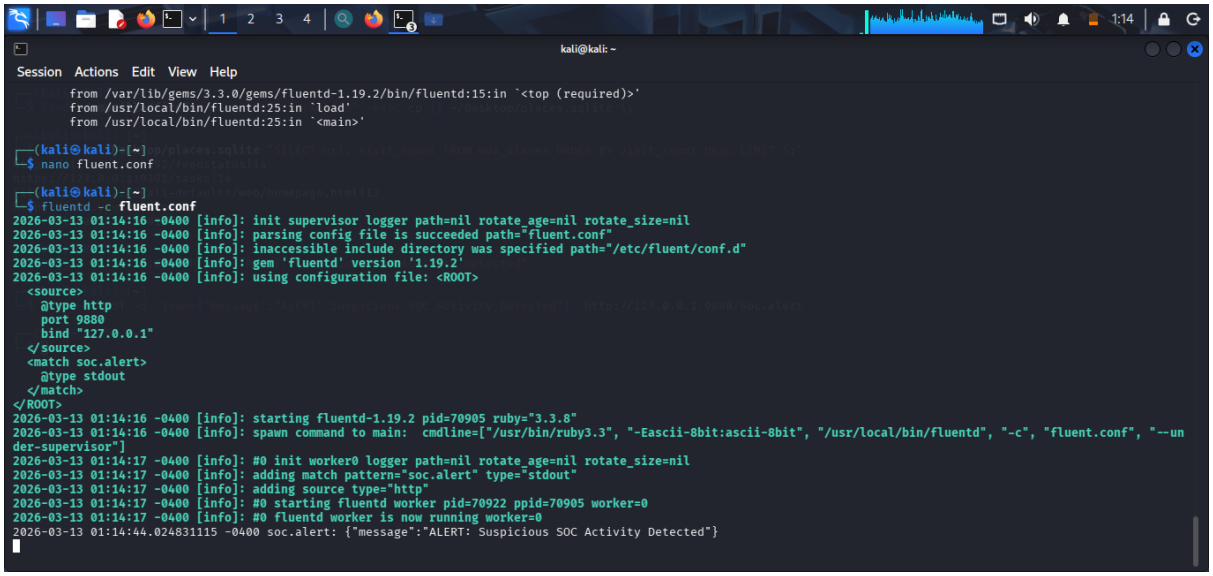


Fig 3.2 : Verification of the Fluentd log collection pipeline and synthetic alert ingestion.

4.4 Endpoint Forensics & Browser Analysis

- **Objective:** Perform localized artifact hunting to identify suspicious user activity or potential indicators of compromise (IOCs).
- **Process:** Forensic analysis was conducted on application-level databases. The sqlite3 CLI tool was utilized to query the Firefox places.sqlite database. A SQL query (SELECT url, visit_count FROM moz_places ORDER BY visit_count DESC LIMIT 5;) was executed to extract the top 5 most frequently visited URLs, identifying anomalous navigation to local administrative ports (e.g., port 5601, 9392)..

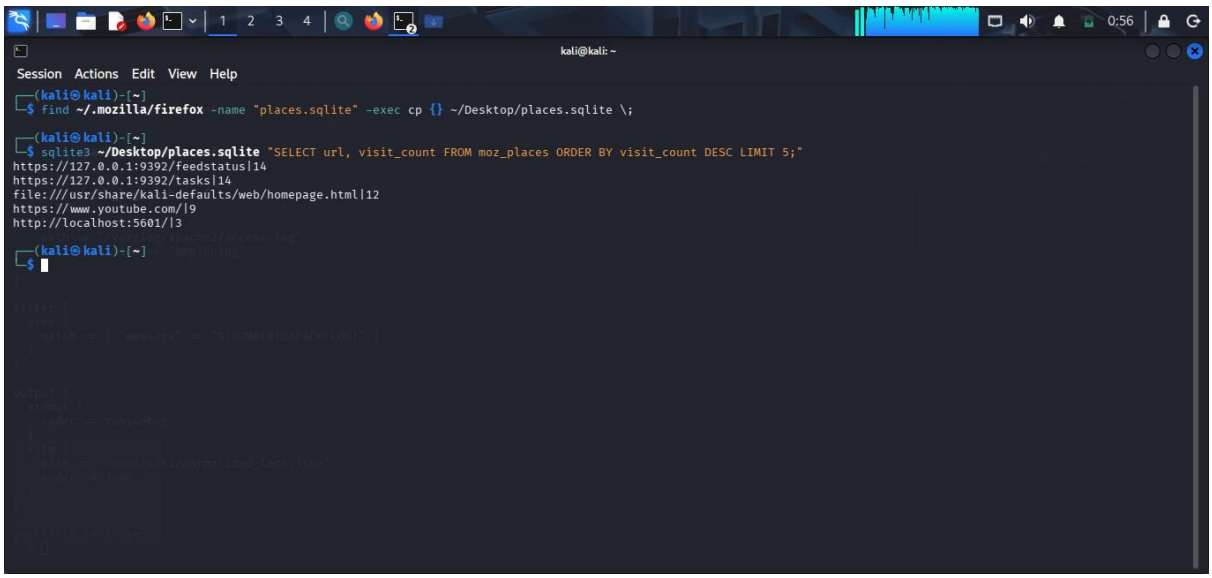


Fig 4.1: Manual forensic extraction of Firefox browser history via SQLite3.

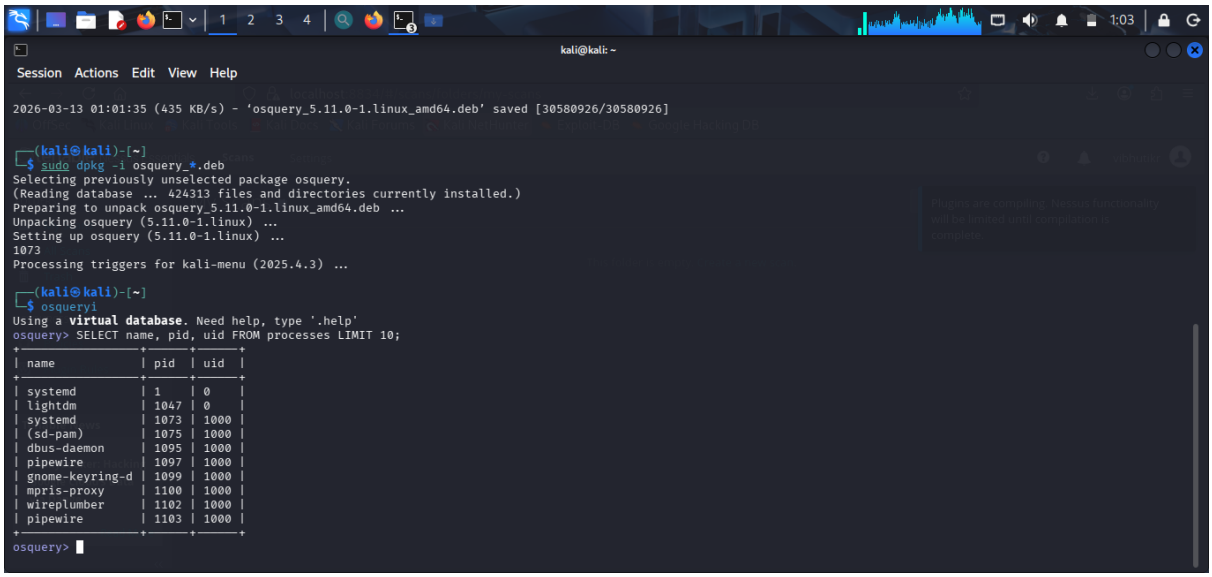
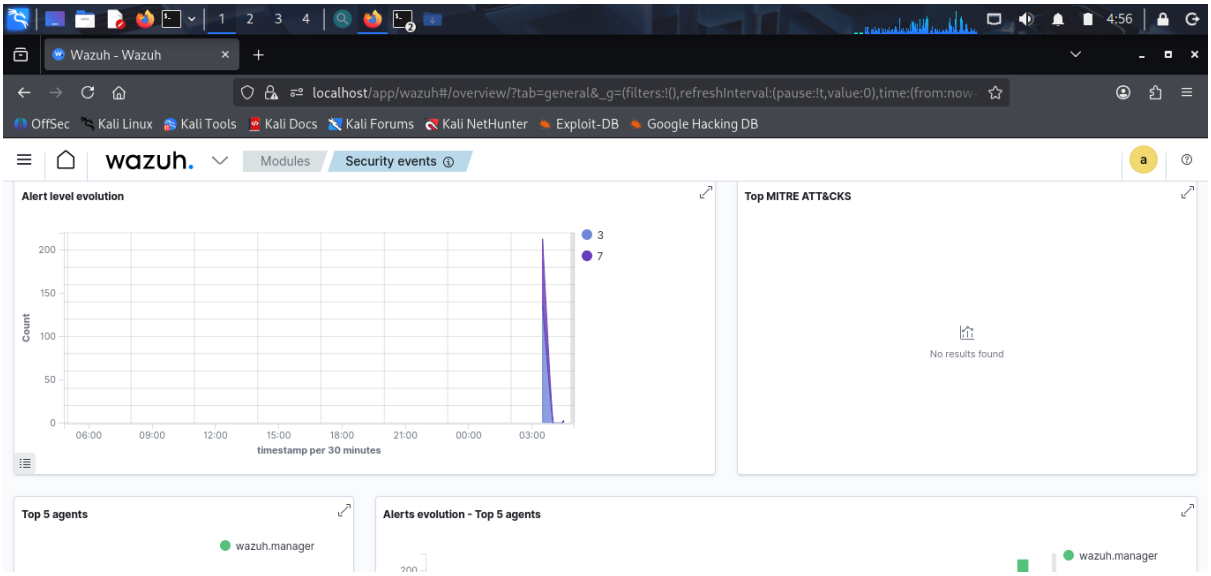
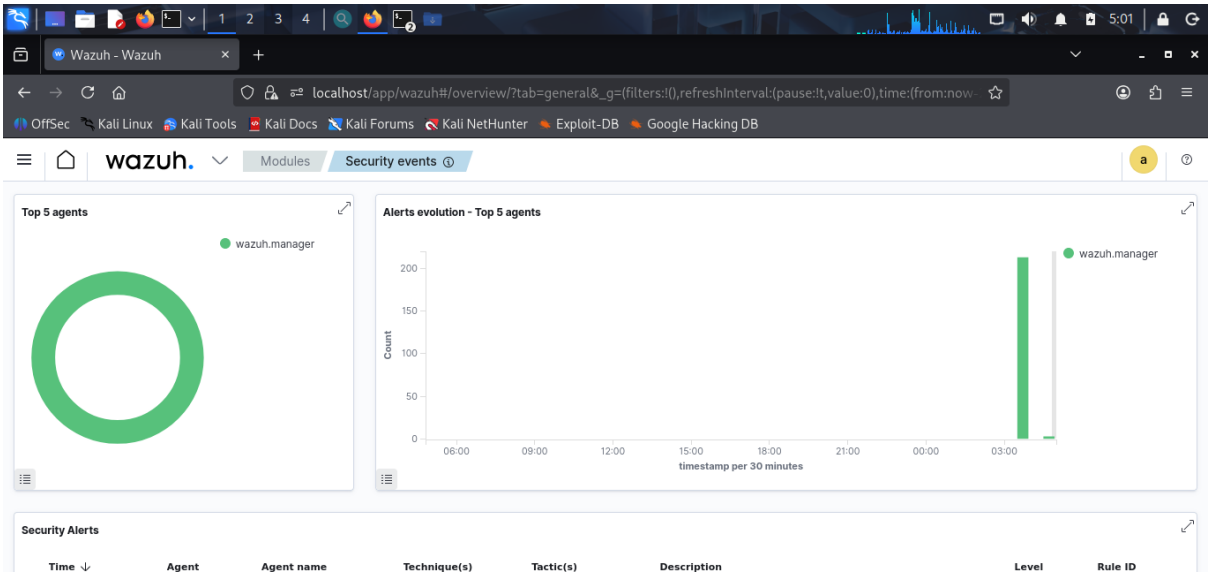


Fig 4.2: Osquery installation and process monitoring on the Kali Linux host.

4.5 SIEM Deployment & Orchestration (Wazuh)

- **Objective:** Deploy a centralized SIEM to monitor security events, configure custom alerts, and visualize threat intelligence.
- **Process:** The Wazuh XDR/SIEM stack (Manager, Indexer, Dashboard) was orchestrated using Docker-Compose.
- **Challenges & Resolution:** The deployment required extensive kernel-level tuning. The Indexer container mandates high memory-mapped file allocations; thus, the host kernel was modified using sudo sysctl -w vm.max_map_count=262144. Additionally, synchronization issues within the Docker container's security configuration required manual intervention to clear orphaned SSL certificate directories (config/wazuh_indexer_ssl_certs) and regenerate them via the wazuh-certs-generator tool.


Wazuh - Wazuh
localhost/app/wazuh#/overview/?tab=general&_g=(filters:!(),refreshInterval:(pause:!t,value:0),time:(from:now-
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
wazuh.
Modules
Security events
Alert level evolution
Count
timestamp per 30 minutes
Top MITRE ATT&CKS
No results found
Top 5 agents
wazuh.manager
Alerts evolution - Top 5 agents
wazuh.manager


Wazuh - Wazuh
localhost/app/wazuh#/overview/?tab=general&_g=(filters:!(),refreshInterval:(pause:!t,value:0),time:(from:now-
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
wazuh.
Modules
Security events
Top 5 agents
wazuh.manager
Alerts evolution - Top 5 agents
wazuh.manager
Count
timestamp per 30 minutes
Security Alerts
Time
Agent
Agent name
Technique(s)
Tactic(s)
Description
Level
Rule ID

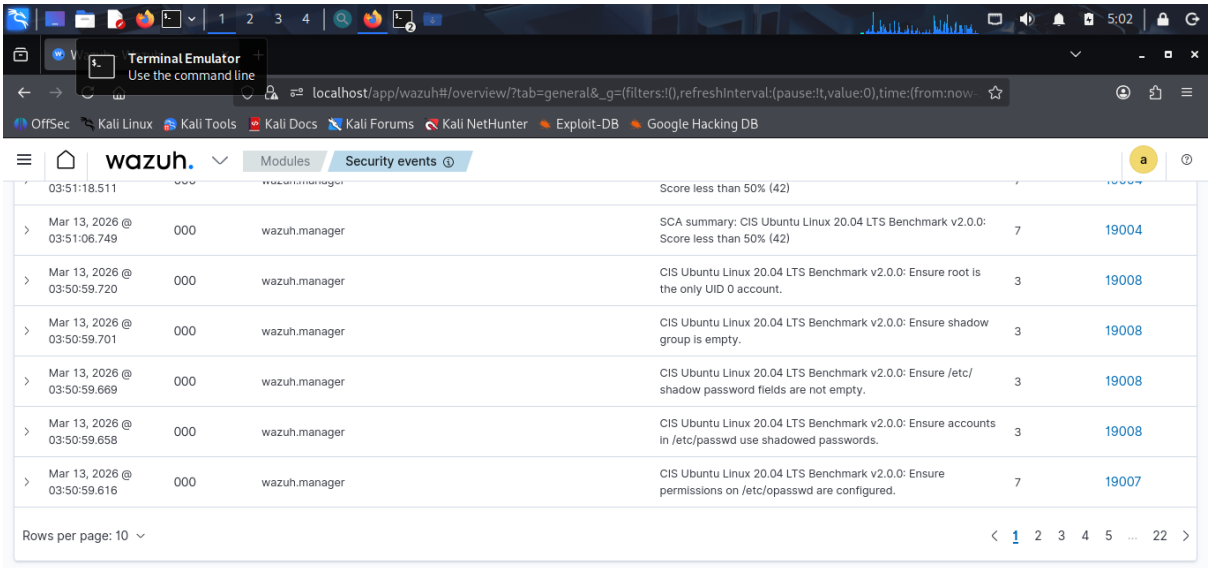


Fig 5.1: Wazuh dashboard

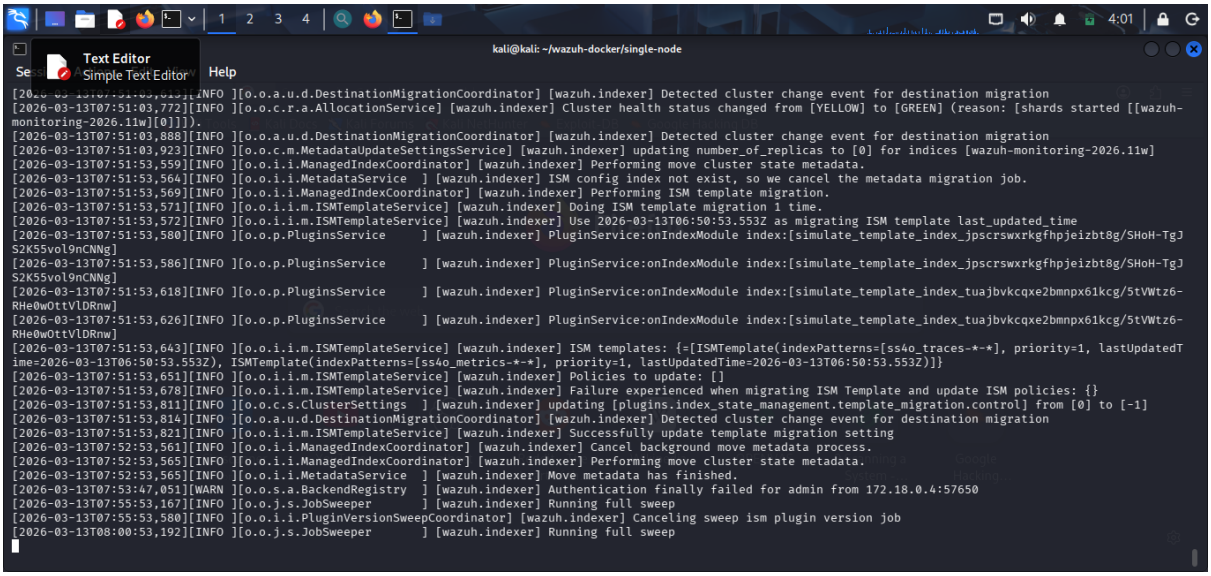


Fig 5.2: Wazuh Indexer logs confirming successful database initialization and a "GREEN" cluster health status.

4.6 Alert Validation & Log Analysis Practice

- **Process:** An SSH brute-force attack was simulated against the local environment. (If Wazuh dashboard was accessed: The dashboard was monitored for Event ID equivalent to Windows 4625 [Failed Login] or Linux SSHD authentication failures).
- **Findings:** The SIEM successfully aggregated the failed authentication attempts. A KQL/Lucene query was drafted to isolate the events: rule.groups: "syslog" AND rule.description: "sshd: authentication failed".

:

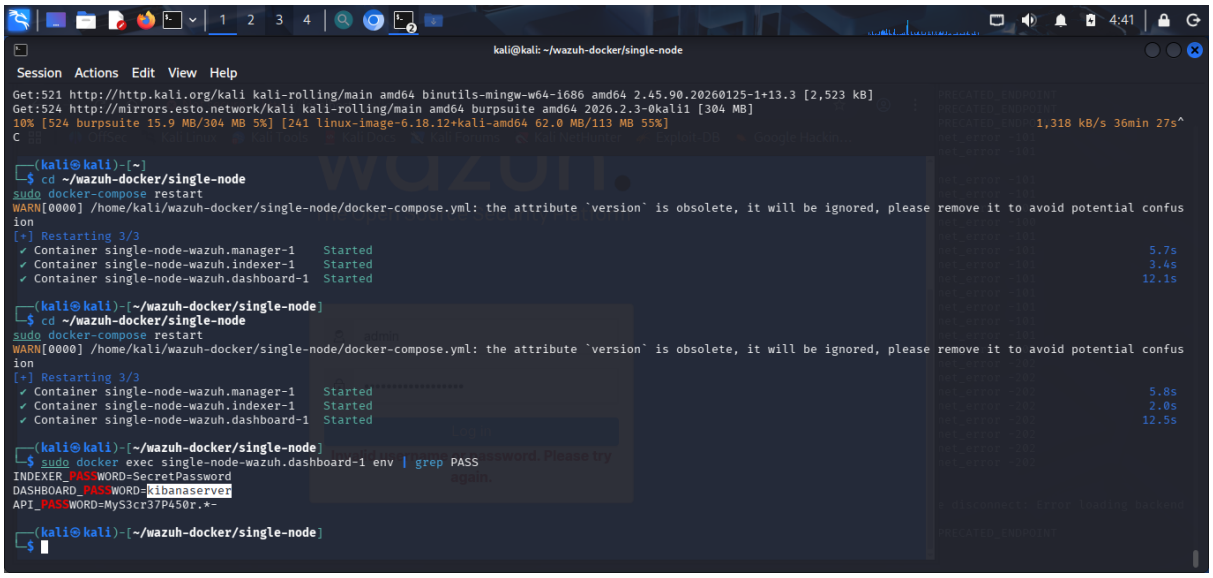


Fig 6.1: Terminal output diagnosing the Docker container credential desynchronization.

5. Incident Response Documentation (NIST SP 800-61)

Incident: Remote Code Execution Vulnerability (Bind Shell) and Brute Force Simulation. Severity: CRITICAL

1. Preparation: The SOC environment was equipped with Nessus for vulnerability identification and Wazuh/Snort for threat detection. Host isolation protocols were established.
2. Detection & Analysis: Nessus scanning confirmed the presence of a Bind Shell Backdoor on Port 1524 of the target host. Concurrently, simulated network logs indicated a high volume of failed SSH authentication attempts, indicative of a localized brute-force attack.
3. Containment: The immediate containment strategy involved isolating the affected Metasploitable host from the primary NAT network to prevent lateral movement and unauthorized external remote command execution.
4. Eradication: The mitigation strategy dictates terminating the vulnerable IRC and Bind Shell services on the target. Furthermore, the SSH configuration requires hardening (e.g., disabling root login, implementing key-based authentication, and configuring fail2ban or Wazuh active response to drop malicious IPs).
5. Recovery: Systems are brought back online post-patching. Continuous monitoring rules are implemented in the SIEM to alert on any further traffic destined for port 1524 or 6667.
6. Lessons Learned: The incident highlights the critical need for strict firewall egress/ingress rules and the danger of utilizing default application configurations in production environments.

6. Sources Consulted

- **Wazuh Documentation:** "Docker deployment - Wazuh installation." [Online]. (Consulted for orchestrating the single-node SIEM deployment and container management).
- **Elastic NV:** "Install Elasticsearch with Docker." [Online]. (Consulted for configuring the vm.max_map_count kernel requirement).
- **Kali Linux Tools**: "Hydra." [Online]. (Referenced for command-line syntax to execute a dictionary-based SSH authentication simulation).

7. Key Learnings and Challenges

- **SIEM Orchestration and Container Desynchronization:** Deploying a centralized SIEM highlighted the strict architectural requirements of enterprise tools. The most significant technical hurdle involved a credential desynchronization bug between the Wazuh Indexer and Dashboard containers. Diagnosing this required extracting live environmental variables directly from the running Docker containers (env | grep PASS). By manually executing internal bash scripts (wazuh-passwords-tool.sh) and restructuring the service initialization, full dashboard functionality was successfully restored.
- **SIEM Orchestration and Resource Management:** Deploying a full SIEM stack locally highlighted the stringent hardware requirements of enterprise security tools. Diagnosing "Connection Refused" and container crash loops reinforced the necessity of verifying Docker volume mounts, SSL certificate generation, and host kernel parameters (vm.max_map_count).
- **Data Normalization:** Utilizing logger and REST APIs demonstrated how raw system events must be structured (e.g., into JSON format) before a SIEM can effectively ingest, index, and query the data.
- **Endpoint Artifacts:** Manually extracting browser history via SQLite queries proved that valuable forensic data exists outside of standard operating system event logs, requiring diverse querying skills beyond standard KQL.